Workshop 4

Maria Lucía Castillo García

Índice de contenido

Tabla de puertos.....1
Propuesta de configuración segura.....1
Respuestas de análisis.....2
Reto: propuesta de Security Group.....2
Evidencia de ejecución de cada actividad.....4
Parte 1.....4
Parte 2.....4

Tabla de puertos

Puerto	Servicio	¿Necesita ser público?	¿Por qué?
22	SSH	No, solo mi IP	Con SSH se controla todo el servidor, entonces solo debe poder entrar quien lo administra.
80	HTTP	Sí	Es por donde entran los usuarios a la página. Lo atiende Nginx.
5000	Flask	No	Solo Nginx necesita hablar con Flask, y lo hace por dentro de Docker.
5432	PostgreSQL	No	Solo la aplicación usa la base de datos. Si queda abierta, la pueden atacar desde afuera.

Propuesta de configuración segura

La idea es abrir solo lo necesario y solo para quien lo necesita:

- Los usuarios entran únicamente por el 80 (y el 443 si hubiera HTTPS), y esa entrada es Nginx.
- El 22 queda abierto solo para mi IP, para poder administrar el servidor.
- Flask (5000) y PostgreSQL (5432) no tienen reglas en el security group, así que desde Internet no se puede llegar a ellos. Se hablan entre ellos por dentro de Docker.

Es la misma configuración que dejé en mi instancia en el Workshop 3: 22 desde mi IP y 80 abierto, sin nada para el 5000 ni el 5432.

Respuestas de análisis

1) ¿Qué riesgo existe al abrir SSH al mundo?

- Que cualquier persona puede intentar entrar. Hay bots que pasan probando el puerto 22 todo el tiempo, y si alguien logra entrar, tiene el control completo del servidor. Restringiéndolo a mi IP, esos intentos ni siquiera llegan.

2) ¿Qué diferencia existe entre Security Group y firewall dentro del sistema operativo?

- El security group lo pone AWS por fuera de la instancia: filtra el tráfico antes de que llegue. El firewall del sistema operativo (como `ufw`) está dentro del servidor. En mi auditoría se ve: el 22 escucha en `0.0.0.0`, o sea que el sistema dejaría entrar a cualquiera, pero el security group solo deja pasar mi IP. Lo mejor es usar los dos.

3) ¿Por qué no debemos exponer PostgreSQL públicamente si no es necesario?

- Porque ahí están los datos, y solo la aplicación necesita conectarse. Si queda abierta, alguien podría adivinar la contraseña o aprovechar una falla y robar o borrar la información.

4) ¿Por qué Nginx puede ser el único punto público de entrada?

- Porque recibe todo por el 80 y él mismo reparte las peticiones a los servicios de atrás, entonces esos servicios no necesitan puertos abiertos. Lo comprobé en el Workshop 3: por el 80 responde y por el 5000 no.

Reto: propuesta de Security Group

Reglas de entrada

Regla	Puerto	Protocolo	Origen	Justificación
HTTP	80	TCP	`0.0.0.0/0`	Entrada de los usuarios a Nginx.
HTTPS	443	TCP	`0.0.0.0/0`	Igual que el 80 pero cifrado. Se agrega cuando haya certificado.
SSH	22	TCP	Mi IP (`/32`)	Solo para administrar el servidor.

Sin regla, o sea cerrados desde Internet: el 5000 (Flask) y el 5432 (PostgreSQL). Las reglas de salida se dejan como vienen, abiertas, para que el servidor pueda descargar actualizaciones e imágenes.

Evidencia de ejecución de cada actividad

Parte 1

sudo ss -tulpn

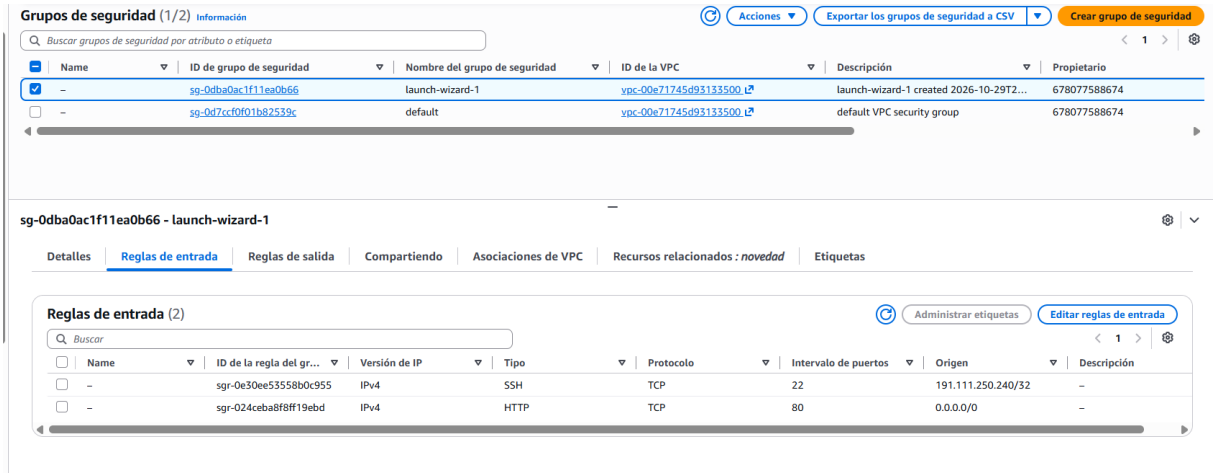
```
ubuntu@ip-172-31-30-115:~/workshop3$ sudo ss -tulpn
Netid          State           Recv-Q          Send-Q                    Local Address:Port                Peer Address:Port         Process
udp            UNCONN          0               0                          127.0.0.54:53                     0.0.0.0:*             users:(("systemd-resolve",pid=197,fd=18))
udp            UNCONN          0               0                       127.0.0.53%lo:53                     0.0.0.0:*             users:(("systemd-resolve",pid=197,fd=16))
udp            UNCONN          0               0                  172.31.30.115%ens5:68                     0.0.0.0:*             users:(("systemd-network",pid=528,fd=36))
udp            UNCONN          0               0                        127.0.0.1:323                      0.0.0.0:*             users:(("chronyd",pid=753,fd=4))
udp            UNCONN          0               0                            [::1]:323                         [::]:*             users:(("chronyd",pid=753,fd=5))
tcp            LISTEN          0               4096                    127.0.0.53%lo:53                     0.0.0.0:*             users:(("systemd-resolve",pid=197,fd=17))
tcp            LISTEN          0               4096                       0.0.0.0:22                        0.0.0.0:*             users:(("sshd",pid=720,fd=3),("systemd",pid=1,fd=243))
tcp            LISTEN          0               4096                       0.0.0.0:80                        0.0.0.0:*             users:(("docker-proxy",pid=10176,fd=8))
tcp            LISTEN          0               4096                    127.0.0.54:53                        0.0.0.0:*             users:(("systemd-resolve",pid=197,fd=19))
tcp            LISTEN          0               4096                          [::]:22                           [::]:*             users:(("sshd",pid=720,fd=4),("systemd",pid=1,fd=246))
tcp            LISTEN          0               4096                          [::]:80                           [::]:*             users:(("docker-proxy",pid=10181,fd=8))
ubuntu@ip-172-31-30-115:~/workshop3$
```

Puerto	Proceso	Qué es
22	`sshd`	SSH. Escucha en todas las interfaces (`0.0.0.0`).
80	`docker-proxy`	Docker pasando el puerto 80 al contenedor de Nginx.
53	`systemd-resolve`	DNS del sistema. Solo escucha en localhost.
68	`systemd-network`	DHCP: así la instancia recibe su IP privada.
323	`chronyd`	Sincroniza la hora. Solo localhost.

El 5000 no aparece porque Flask solo existe dentro de la red de Docker, y el 5432 tampoco porque no instalé PostgreSQL.

Parte 2

Reglas de entrada del security group



Reglas de salida del security group

